

Week 3: DAY-12 Hands On – Keerthivasan R

ANSI SQL using MS-SQL Server

Level-1 and 2 Problem: Create Database **EventDb** and create tables as per schema given below.

🔴 Requirements

1. Add entity class **UserInfo** and add public properties:

Property Name	Type	Constraint
EmailId	varchar	PK
UserName	varchar	Not Null, Length: 1 to 50 char.
Role	varchar	Not Null, (Admin,Participant)-CHECK
Password	varchar	Not Null Length: 6 to 20 char

QUERY:

```
CREATE DATABASE EventDb;  
GO
```

```
USE EventDb;  
GO
```

```
CREATE TABLE UserInfo (  
    EmailId VARCHAR(255) PRIMARY KEY,  
  
    UserName VARCHAR(50) NOT NULL  
        CONSTRAINT CK_UserName_Length CHECK (LEN(UserName) >= 1),  
  
    [Role] VARCHAR(20) NOT NULL  
        CONSTRAINT CK_UserRole CHECK ([Role] IN ('Admin', 'Participant')),  
  
    [Password] VARCHAR(20) NOT NULL  
        CONSTRAINT CK_Password_Length CHECK (LEN([Password]) BETWEEN 6 AND 20)  
);
```

OUTPUT:

Commands completed successfully.

Completion time: 2026-03-03T21:36:17.8087905+05:30

2. Add entity class **EventDetails** and add below public properties.

Property Name	Type	Constraint
EventId	int	PK
EventName	varchar	Not Null, Length: 1 to 50 char.
EventCategory	varchar	Not Null, Length: 1 to 50 char.
EventDate	datetime	Not Null
Description	varchar	Nullable
Status	varchar	Active or In-Active, CHECK

QUERY:

```
USE EventDb;
GO

CREATE TABLE EventDetails (
    -- EventId as Primary Key
    EventId INT PRIMARY KEY,

    -- EventName: 1 to 50 characters
    EventName VARCHAR(50) NOT NULL
        CONSTRAINT CK_EventName_Length CHECK (LEN(EventName) >= 1),

    -- EventCategory: 1 to 50 characters
    EventCategory VARCHAR(50) NOT NULL
        CONSTRAINT CK_EventCategory_Length CHECK (LEN(EventCategory) >= 1),

    -- EventDate: Standard datetime
    EventDate DATETIME NOT NULL,

    -- Description: Explicitly Nullable (no length constraint provided)
    [Description] VARCHAR(MAX) NULL
);
GO
```

OUTPUT:

Commands completed successfully.

Completion time: 2026-03-03T21:40:30.5740292+05:30

3. Add entity class **SpeakersDetails** and add below public properties.

Property Name	Type	Constraints
SpeakerId	int	PK
SpeakerName	varchar	Not Null, Length: 1 to 50 char.

QUERY:

```
USE EventDb;
GO

CREATE TABLE SpeakersDetails (
    -- SpeakerId as Primary Key
    SpeakerId INT PRIMARY KEY,

    -- SpeakerName: Mandatory, between 1 and 50 characters
    SpeakerName VARCHAR(50) NOT NULL
        CONSTRAINT CK_SpeakerName_Length CHECK (LEN(SpeakerName) >= 1)
);
GO
```

OUTPUT:

Commands completed successfully.

Completion time: 2026-03-03T21:44:29.6009399+05:30

4. Add entity class **SessionInfo** and add below public properties.

Property Name	Type	Constraints
SessionId	int	PK
EventId	int	Not Null (FK)
SessionTitle	varchar	Not Null, Length: 1 to 50 char.
SpeakerId	int	Not Null (FK)
Description	varchar	Nullable
SessionStart	datetime	Not Null (Time Slot)
SessionEnd	datetime	Not Null (Time Slot)
SessionUrl	varchar	

QUERY:

```
USE EventDb;
GO

CREATE TABLE SessionInfo (
    -- SessionId as Primary Key
    SessionId INT PRIMARY KEY,

    -- EventId as Foreign Key (linking to EventDetails)
    EventId INT NOT NULL,

    -- SessionTitle: 1 to 50 characters
    SessionTitle VARCHAR(50) NOT NULL
        CONSTRAINT CK_SessionTitle_Length CHECK (LEN(SessionTitle) >= 1),

    -- SpeakerId as Foreign Key (linking to SpeakersDetails)
    SpeakerId INT NOT NULL,

    -- Description: Nullable
    [Description] VARCHAR(MAX) NULL,

    -- Time Slots: Mandatory datetime fields
    SessionStart DATETIME NOT NULL,
    SessionEnd DATETIME NOT NULL,

    -- SessionUrl: Nullable (assumed based on standard event patterns)
    SessionUrl VARCHAR(2048) NULL,

    -- Foreign Key Constraints
    CONSTRAINT FK_Session_Event FOREIGN KEY (EventId)
        REFERENCES EventDetails(EventId),

    CONSTRAINT FK_Session_Speaker FOREIGN KEY (SpeakerId)
        REFERENCES SpeakersDetails(SpeakerId),

    -- Logical Constraint: Session cannot end before it starts
    CONSTRAINT CK_SessionTime CHECK (SessionEnd > SessionStart)
);
GO
```

OUTPUT:

Commands completed successfully.

Completion time: 2026-03-03T21:46:26.1061266+05:30

5. Add entity class **ParticipantEventDetails** and add below public properties.

Property Name	Type	Constraints
Id	int	(PK)
ParticipantEmailId	varchar	Not Null (FK)
EventId	int	Not Null (FK)
SessionId	Int	Not Null (FK)
IsAttended	bit	Yes or No, CHECK

Technical Constraints

- Use SQL Server.
- Use appropriate data types (INT, VARCHAR, DATE, DECIMAL).
- Follow normalization up to 2NF.
- Use ANSI SQL syntax only.

Expectations:

- Proper table structure with relationships.
- Valid constraints implementation.
- Correct data insertion and retrieval queries.

Learning Outcome

You will understand database structure, basic normalization, constraints, and ANSI SQL queries.

Query:

```
USE EventDb;  
GO
```

```
CREATE TABLE ParticipantEventDetails (  
    -- Id as Primary Key  
    Id INT PRIMARY KEY,  
  
    -- ParticipantEmailId as Foreign Key (linking to UserInfo.EmailId)  
    ParticipantEmailId VARCHAR(255) NOT NULL,  
  
    -- EventId as Foreign Key (linking to EventDetails)  
    EventId INT NOT NULL,  
  
    -- SessionId as Foreign Key (linking to SessionInfo)
```

```

SessionId INT NOT NULL,

-- IsAttended: bit (0 for No, 1 for Yes)
-- The CHECK constraint ensures only 0 or 1 is entered
IsAttended BIT NOT NULL
    CONSTRAINT CK_Attendance_Values CHECK (IsAttended IN (0, 1)),

-- Foreign Key Constraints
CONSTRAINT FK_Participant_User FOREIGN KEY (ParticipantEmailId)
    REFERENCES UserInfo(EmailId),

CONSTRAINT FK_Participant_Event FOREIGN KEY (EventId)
    REFERENCES EventDetails(EventId),

CONSTRAINT FK_Participant_Session FOREIGN KEY (SessionId)
    REFERENCES SessionInfo(SessionId)
);
GO

INSERT RECORDS:

-- 1. Insert User
INSERT INTO UserInfo (EmailId, UserName, Role, Password)
VALUES ('john.doe@example.com', 'JohnDoe', 'Participant', 'SecurePass123');

-- 2. Insert Event
INSERT INTO EventDetails (EventId, EventName, EventCategory, EventDate, Description)
VALUES (101, 'Tech Summit 2026', 'Technology', '2026-05-15 09:00:00', 'Annual dev
conference');

-- 3. Insert Speaker
INSERT INTO SpeakersDetails (SpeakerId, SpeakerName)
VALUES (501, 'Dr. Aris AI');

-- 4. Insert Session
INSERT INTO SessionInfo (SessionId, EventId, SessionTitle, SpeakerId, SessionStart,
SessionEnd, SessionUrl)
VALUES (2001, 101, 'Future of SQL', 501, '2026-05-15 10:00:00', '2026-05-15 11:00:00',
'http://meet.tech.com/sql');

-- 5. Insert Participation Record
INSERT INTO ParticipantEventDetails (Id, ParticipantEmailId, EventId, SessionId,
IsAttended)
VALUES (1, 'john.doe@example.com', 101, 2001, 1);

RETRIEVE:

SELECT
    U.UserName,
    E.EventName,
    S.SessionTitle,
    SD.SpeakerName,
    CASE WHEN P.IsAttended = 1 THEN 'Yes' ELSE 'No' END AS Attended
FROM ParticipantEventDetails P
JOIN UserInfo U ON P.ParticipantEmailId = U.EmailId
JOIN EventDetails E ON P.EventId = E.EventId
JOIN SessionInfo S ON P.SessionId = S.SessionId
JOIN SpeakersDetails SD ON S.SpeakerId = SD.SpeakerId;

```

OUTPUT:

	UserName	EventName	SessionTitle	SpeakerName	Attended
1	JohnDoe	Tech Summit 2026	Future of SQL	Dr. Aris AI	Yes